

BRIEF 3

AuditDB

1. Du brief 2 au brief 3

À la fin du brief 2, LogAnalyzer fonctionne : il charge tous les logs **en mémoire** et les analyse avec les Streams. Tout est stocké dans des **fichiers texte** (files.txt, access.log). Mais une question a émergé…

LA PROBLÉMATIQUE

Que se passe-t-il quand les données deviennent **volumineuses** (des millions de lignes) et **interrogées de multiples façons** ?

- **Tout charger en mémoire** → la mémoire sature (OutOfMemoryError).
- **Relire le fichier à chaque question** → beaucoup trop lent.
- Les données **ne survivent pas** à l'arrêt du programme et ne sont **pas partageables**.

LA SOLUTION

Ne plus gérer les données en mémoire, mais les confier à une **base de données relationnelle**. On stocke users, fichiers et logs dans des tables SQL, et on **interroge** l'historique avec des requêtes — sans jamais tout charger. C'est le rôle de **AuditDB**.

Ce brief poursuit LinePerm : les mêmes concepts métier (utilisateurs, fichiers, journaux d'accès), mais avec un **stockage professionnel** et une **architecture objet avancée**.

2. Objectifs du projet

CONTEXTE

Construire **AuditDB**, une application console qui gère en base de données les **utilisateurs**, les **fichiers** et les **journaux d'accès**, avec un CRUD complet et des requêtes d'audit.

2.1 — Compétences techniques (JDBC)

- Se connecter à une base avec **JDBC** (DriverManager, Connection).
- Exécuter des requêtes avec **PreparedStatement**.
- Réaliser un **CRUD** et écrire des **jointures** entre tables.

2.2 — Concepts POO avancés

CONCEPTS POO MOBILISÉS

- **Encapsulation** — attributs private dans les modèles (User, Fichier, Log) + getters/setters.
- **Abstraction** — une interface Dao<T> (le contrat) et une classe abstraite AbstractDao<T> (le code commun).
- **Héritage** — chaque DAO concret extends AbstractDao et réutilise la connexion.

- **Polymorphisme** — manipuler un Dao<User> sans savoir quelle implémentation concrète se cache derrière.

3. Le schéma de la base (3 tables)

Trois tables reliées : users, fichiers, logs. Un utilisateur possède des fichiers ; chaque log référence un utilisateur et un fichier.

Règles de gestion:

users

- Un utilisateur est identifié de façon unique (id) ; son login est unique (pas deux utilisateurs de même nom).
- Login et mots de passe sont obligatoires (non nuls).

fichiers

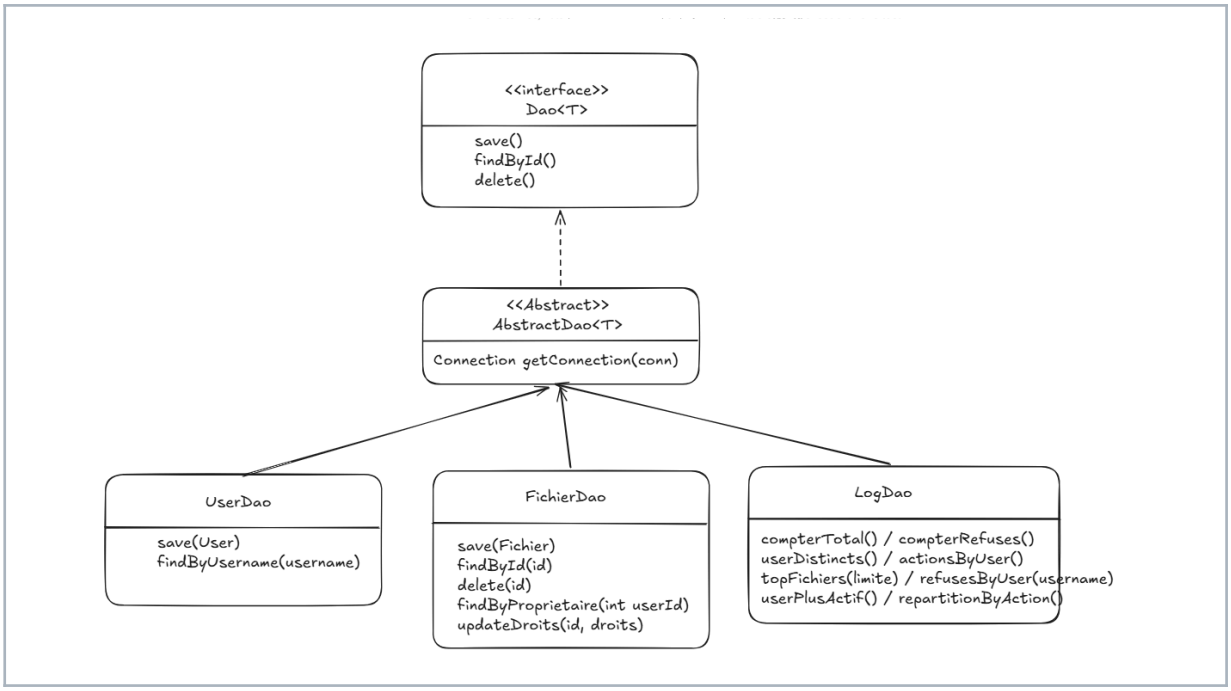
- Un fichier est identifié de façon unique (id).
- Un fichier appartient à un et un seul propriétaire (un utilisateur).
- Un fichier porte des droits et un nom, obligatoires.

logs

- Un log est identifié de façon unique (id).
- Un log enregistre un accès : qui (utilisateur), quel fichier, quelle action, quel résultat, et quand.
- Chaque log référence obligatoirement un utilisateur **et** un fichier.
- Un log est immuable : une fois écrit, il n'est ni modifié ni supprimé (nature d'un journal d'audit).

4. L'architecture orientée objet

Organiser le code avec les **4 piliers de la POO**. Le schéma ci-dessous montre comment ils s'articulent autour du Pattern **DAO** (Data Access Object).



Singleton : une connexion unique (DBConnection)

Le pattern Singleton garantit qu'il n'existe qu'une seule instance de DBConnection dans toute l'application, qui détient une connexion unique utilisée partout.

5. Fonctionnalités demandées

5.1 — Utilisateurs (CRUD via UserDao)

- **Ajouter**, **rechercher par login**, **supprimer** un utilisateur.

5.2 — Fichiers (via FichierDao)

- **Enregistrer** un fichier (propriétaire + droits), **lister par propriétaire**, **modifier les droits**.

5.3 — Journal d'accès et audit (via LogDao)

- Enregistrer chaque accès (qui, quel fichier, action, résultat).
- Nombre total d'actions
- Nombre d'accès refusés
- Utilisateurs distincts
- Actions par utilisateur
- Top 3 des fichiers consultés
- Accès refusés d'un utilisateur
- Utilisateur le plus actif
- Répartition des actions par type

6. Gestion des Exceptions

Une application connectée à une base rencontre forcément des erreurs : serveur arrêté, doublon, saisie invalide, id inexistante. Bien les gérer, ce n'est pas juste afficher la trace dans un catch : c'est choisir, selon le cas, de réessayer, d'avertir l'utilisateur ou d'arrêter proprement

- Erreurs de connexion (anormales)
- Violations de contraintes SQL (attendues)
- Résultats de requêtes
- Saisies de l'utilisateur (console)

Pour aller plus loin (bonus)

- Importer l'ancien access.log (brief 2) dans la table logs.
- Ajouter des **transactions** (commit / rollback).